The 19th Workshop on Combinatorial Mathematics and Computation Theory (Algo2002)

A Routing Scheme for Constructing Node-to-Node Disjoint Paths in Alternating Group Graphs

Chin-Tsai Lin (林錦財)*

Assistant Professor
Department of Information Management,
Kun-Shan University of Technology

Wen-Chuan Chiu (邱文泉)

Graduate Student
Department of Industrial Technology Education,
National Kaohsiung Normal University

Abstract

The paper illustrates a simple scheme for constructing a maximal set of node-to-node disjoint paths in an alternating group graph. Among the disjoint paths, there is at least one shortest path and the length of the longest one(s) is no more than that of a shortest path by four.

Keywords: Interconnection networks, Cayley graphs, alternating group, node-disjoint paths

*Correspondance: 台南縣永康市 710 忠義街 10 巷 1 號, Taiwan
Tel.: (06) 311-2107 (0921) 217-922
Fax.: (06) 273-2726
E-mail: ctlin@mail.ksut.edu.tw
jench1@seed.net.tw

A Routing Scheme for Constructing Node-to-Node Disjoint Paths in Alternating Group Graphs

Chin-Tsai Lin

Department of Information Management, Kun-Shan University of Technology

Wen-Chuan Chiu

Department of Industrial Technology Education, National Kaohsiung Normal University

Taiwan, Republic of China

Abstract

The paper illustrates a simple scheme for constructing a maximal set of node-to-node disjoint paths in an alternating group graph. Among the disjoint paths, there is at least one shortest path and the length of the longest one(s) is no more than that of a shortest path by four.

Keywords: Interconnection networks, Cayley graphs, alternating group, node-disjoint paths

1. Introduction

Cayley graphs provide a rich framework for the design of the topology of interconnection networks. Famous network topologies such as hypercube[4], star graphs[1], etc. are able to be modeled as Cayley graphs[2]. Jwo, Lakshmivarahan and Dhall [13] proposed Cayley graphs based on the alternating groups. They showed that alternating group

graphs are edge symmetry, 2-transitive, Hamiltonian and strongly hierarchical. They also described algorithms for embedding a class of multidimensional grids and a variety of cycles in an alternating group graph. With respect to contention problem for general routing, it has been shown that alternating group graphs perform better than the star graphs and are close to the hypercube.

Alternating group graphs have received considerable attention in [4,5,10,15,17,19]. In [15], Lai and Tsay have presented algorithm for all-port all-to-all broadcasting and single-node scattering whose time performances are one step more than the lower bounds. Hung and Huang [10] also introduced an optimal one-port one-to-all broadcasting algorithm on alternating group graphs. Ho, et. al.[19] introduced a method to explore a set of edge-disjoint paths between any two nodes. In [17], it has been shown that the fault diameter of an alternating group graph is no more than the original diameter by one.

The alternating group graphs are defined as follows. Let $<n>=\{1, 2, \ldots, n\}$, $p = p_1p_2\ldots p_n$, $p_i \in <n>$ and $p_i \neq p_j$ for $i \neq$ j, where p_i denotes the element at position i for $1 \leq i \leq n$. That is, p is a permutation of $<n>$. The permutation p can also be represented by its cycle structure as

$$p = c_1c_2\ldots c_ke_1e_2\ldots e_l,$$

where c_i is a cycle of length $|c_i| \geq 2$ for $1 \leq i \leq k$ and e_i is an invariant for $1 \leq i \leq l$. Thus, $n = \sum_{i=1}^{k} |c_i| + l$. To simplify the notation, we may omit the set of all invariants in the cycle structure of p. For example, the cycle structure of permutation 132546 is (23)(45)(1)(6),

where (1) and (6) are invariants and may be omitted. Let $\zeta(p)$ denote the number of inversions in p. The parity of p is defined as $\eta(p) = (-1)^{\zeta(p)}$. A permutation is called *even* or *odd* depending on its parity being +1 or –1. Thus, 132546 is an even permutation.

Let S_n be the symmetric group, i.e., S_n contains all the permutations of n elements. The alternating group A_n contains the set of all even permutations of S_n, where $|A_n| = n!/2$. Let $g_i^+ = (1\ 2\ i)$, $g_i^- = (1\ i\ 2)$ and $\Omega = \{g_i^+ \mid 3 \le i \le n\} \cup \{g_i^- \mid 3 \le i \le n\}$. It is well known that Ω is a generator set for A_n.

Definition 1. An alternating group graph of dimension n, $AG_n = (V_n, E_n)$, is defined as

$$V_n = A_n, \text{ the set of all even permutations of } <n>$$

and

$$E_n = \{(p, q) \mid p, q \in A_n, q = p \circ h, \text{ for } h \in \Omega\},$$

where "$\circ$" is the usual binary combination operator defined as $f \circ g(x) = f(g(x))$.

Let $(1\ 2)\cdot(AG_n)$ denote the dual graph of AG_n in which the set of nodes is $\{(1\ 2)\cdot p \mid p \in A_n\}$ and the set of edges is $E_n = \{(p, q) \mid (1\ 2)\cdot p, (1\ 2)\cdot q \in A_n, q = p \circ h, \text{ for } h \in \Omega\}$. It can be verified that $(1\ 2)\cdot(AG_n)$ is isomorphic to AG_n.

An important parameter for graphs is their *fault toleranc*e. The fault tolerance of a graph is defined as the maximum number of nodes that can be removed from it provided that the

remaining graph is still connected. Hence, the fault tolerance of a graph is defined to be one less than its connectivity. According to the results in [5], the connectivity of the alternating group graphs is optimal, i.e., equal to the degree.

In this paper, we will introduce a scheme for constructing a maximal set of node-disjoint paths between any two nodes in an alternating group graph. The paper is organized as follows. In the next section, we will introduce the basic properties of alternating group graphs. Section 3 discusses the node-disjoint routing algorithm. Finally, concluding remarks and some directions of further researches direction are exhibited.

2. Background

The n dimensional alternating group graph AG_n is a regular graph with degree $2(n-2)$, $|V_n| = n!/2$, $|E_n| = (n-2)\times n!/2$, diameter $d_n = \lfloor 3\times(n-2)/2 \rfloor$, and the connectivity is $2(n-2)$. Alternating group graphs have a highly recursive structure. AG_n is made up of n copies of AG_{n-1}.

An understanding of the routing algorithm is essential for our development. Let $a, b \in A_n$. By definition, $(a, b) \in E_n$ if and only if $b=a\circ h$, $h \in \Omega$. By extending the notion of an edge in AG_n, the path from a to b can be represented by a sequence of generators:

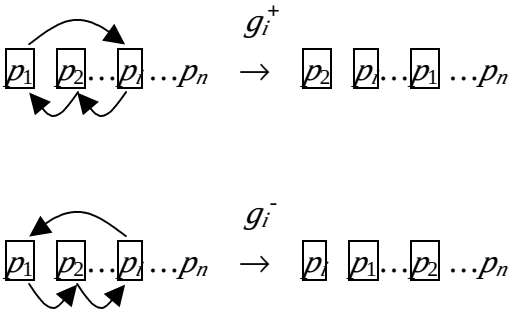
Path$(a, b) = h_1 \cdot h_2 \cdots h_t$,

that is, $b = a \cdot h_1 \cdot h_2 \cdots h_t$, where $h_i \in \Omega$ for $1 \le i \le t$. By group theory, we have

$I = b^{-1} \cdot a \cdot h_1 \cdot h_2 \cdots h_t$.

Let $p = b^{-1} \cdot a$. Then, the properties of path from a to b can be analyzed by considering the path from p to I. So we can focus on how to route from p to I only, and the routing is equivalent to sorting a permutation.

The following diagram shows what is changed by a routing step on g_i^+ and g_i^-, $3 \le i \le n$.



As shown in the diagram, g_i^+ and g_i^- rotate the three symbols at position 1, 2 and i with different directions: g_i+ moves the symbol at position i to position 2, the symbol at position 2 to position 1, and the symbol at position 1 to position i; while g_i^- moves the symbol at position i to position 1, the symbol at position 1 to position 2, and the symbol at position 2 to position i. It can be verified that $g_i^+ \cdot g_i^+ = g_i^-$, $g_i^- \cdot g_i^- = g_i^+$, $(g_i^+)^{-1} = g_i^-$ and $(1\ 2) \cdot g_i^+ \cdot (1\ 2) = g_i^-$, for $3 \le i \le n$. To find the shortest path from p to I is to optimally *sort* the set of symbols in p using the basic rotations. (Symbol i is sorted if it is at position i.) It should be noticed that there is no need to sort symbol 1 and 2. They will be automatically sorted after sorting symbol

3, 4, …, n because it should be an even permutation.

The shortest path routing algorithm from $p = p_1p_2\ldots p_n$ to $q = q_1q_2\ldots q_n$ may be described as follows:

Algorithm 1. route(p, q) { // p and q are two nodes in AG_n, finding a shortest path from p to q.

$p' = p$; // the current node

while ($p' \neq q$) {

if ($p'_1 \notin \{q_1, q_2\}$) {

$i = p'_1$;

output(g_i^+);

$p' = p' \circ g_i^+$;

} else if ($p'_2 \notin \{q_1, q_2\}$) {

$i = p'_2$;

output(g_i^-);

$p' = p' \circ g_i^-$;

} else { // $p'_1, p'_2 \in \{q_1, q_2\}$

Select a non-invariant i, such that $p'_i \neq q_i$ and $3 \leq i \leq n$;

output(g_i^+); $p' = p' \circ g_i^+$; // If p'_1 should move out of positions 1 and 2

// output(g_i^-); $p' = p' \circ g_i^-$; // If p'_2 should move out of positions 1 and 2

}

} // end of while

} // end of algorithm

As an example, let p = 14523 = (2 4)(5 3). A shortest path from p to I is as follows:

$$14523\ g_4^-\ \ 21543\ g_3^+\ \ 15243\ g_5^-\ \ 31245\ g_3^+\ \ 12345.$$

Let D_p denote the length measured in terms of the number of edges in the shortest path from p to I. We have the following lemma [13].

Lemma 2. If $p = c_1c_2\ldots c_k e_1 e_2 \ldots e_l \in A_n$, then

$D_p = n+k-l$ if $p_1 = 1$ and $p_2 = 2$

$= n+k-l-3$ if $p_1 = 2$ and $p_2 = 1$

$= n+k-l-2$ if $p_1 \neq 1$ and $p_2 = 2$

$= n+k-l-2$ if $p_1 = 1$ and $p_2 \neq 2$

$= n+k-l-3$ if $1, 2 \in c_i$ for some i and $|c_i| \geq 3$

$= n+k-l-4$ if $1 \in c_i$ and $2 \in c_j$, $i \neq j$.

The result of routing is obtained in the form of a product of generators. Since the product of two generators of alternating group graphs is not commutative, there is no trivial rule for finding alternative paths of the same length. However, like star graphs [11], rules for finding the alternative paths in an alternating group graph can be based on the concepts of “ordinary” and “barrel” products of generators. Ordinary and barrel products are basic parts of any shortest path that can be manipulated in order to obtain alternative paths. The whole

analysis relies on the mapping of each generator to the corresponding movement of symbols. Therefore, the complex product can be tracked as a set of movements of symbols.

Let $\sigma(i)$ denote the *sign* + if i is even, or the sign - if i is odd.

Definition 3. The *initial product* is the product of the following form: for some integer l,

$$h_1^{\sigma(l)} h_2^{\sigma(l+1)} \cdots h_n^{\sigma(l+n-1)}.$$

Definition 4. The *ordinary product* is any product that is an arbitrary permutation of an arbitrary subset of k distinct-index generators with alternating signs from the initial product.

Definition 5. The *barrel product* of generators is obtained when the first generator of an ordinary product is appended at the end of that product with alternating signs, i.e., the product of the following form:

$$h_1^{\sigma(i)} h_2^{\sigma(i+1)} \cdots h_n^{\sigma(i+n-1)} h_1^{\sigma(i+n)}.$$

For alternating group graphs, applying a barrel product of generators on I will generate an even permutation containing one cycle without symbols 1 and 2, while applying an ordinary product on I will generate an even permutation containing one cycle with a symbol 1 or 2. The properties of the barrel product are exhibited in the following lemmas and its corollaries.

Lemma 6. For AG_n, the following holds. (Inverting signs)

$$g_3^{+}g_4^{-}\cdots g_{n-2}^{\sigma(n-3)}g_{n-1}^{\sigma(n-2)}g_n^{\sigma(n-1)}g_3^{\sigma(n)}$$

$$= g_3^{-}g_4^{+}\cdots g_{n-2}^{\sigma(n-2)}g_{n-1}^{\sigma(n-1)}g_n^{\sigma(n)}g_3^{\sigma(n+1)}.$$

Lemma 7. For AG_n, the following holds. (Rotation)

$$g_3^{+}g_4^{-}\cdots g_{n-2}^{\sigma(n-3)}g_{n-1}^{\sigma(n-2)}g_n^{\sigma(n-1)}g_3^{\sigma(n)}$$

$$= g_4^{-}g_5^{+}\cdots g_{n-1}^{\sigma(n-4)}g_n^{\sigma(n-3)}g_3^{\sigma(n-2)}g_4^{\sigma(n-1)}$$

…

$$= g_{n-1}^{\sigma(n-2)}g_n^{\sigma(n-1)}g_3^{\sigma(n)}g_4^{\sigma(n+1)}\cdots g_{n-3}^{\sigma(2n-6)}g_{n-2}^{\sigma(2n-5)}g_{n-1}^{\sigma(2n-4)}.$$

Corollary 8. For AG_n, let $t_1, t_2, \ldots, t_m$ be distinct integers in $\langle n\rangle - \{1, 2\}$. Then,

$$g_{t_1}^{+}g_{t_2}^{-}\cdots g_{t_m}^{\sigma(m-1)}g_{t_1}^{\sigma(m)}$$

$$= g_{t_1}^{-}g_{t_2}^{+}\cdots g_{t_m}^{\sigma(m)}g_{t_1}^{\sigma(m+1)}$$

$$= g_{t_2}^{+}\cdots g_{t_m}^{\sigma(m-2)}g_{t_1}^{\sigma(m-1)}g_{t_2}^{\sigma(m)}$$

$$= \ldots$$

$$= g_{t_m}^{+}g_{t_1}^{-}\cdots g_{t_{m-1}}^{\sigma(m-1)}g_{t_m}^{\sigma(m)}.$$

That is, Lemmas 6 and 7 hold when an arbitrary ordinary product is substituted for the

ordinary part of barrel product. A barrel product of k distinct-index generators can be represented in $2k$ different ways, all preserving the same cyclic ordering of generators.

Definition 9. Let $\prod = h_1^{\sigma(l)} h_2^{\sigma(l+1)} \cdots h_m^{\sigma(l+m-1)}$ be an ordinary or barrel product. The *invert-sign product* $\prod^-$ of $\prod$ is $h_1^{\sigma(l+1)} h_2^{\sigma(l+2)} \cdots h_m^{\sigma(l+m)}$. The *positive-sign product* $\prod^+$ of $\prod$ is $\prod$ itself.

The following two lemmas introduce the law of commutativity between the ordinary and barrel subproducts of generators.

Lemma 10. For AG_n, let $t_1, t_2, \ldots, t_m$ be distinct integers in $<n> - \{1, 2\}$. Then,

$$g_{t_1}^{+} g_{t_2}^{-} \cdots g_{t_j}^{\sigma(j-1)} g_{t_1}^{\sigma(j)} \boxed{g_{t_{j+1}}^{\sigma(j+1)} g_{t_{j+2}}^{\sigma(j+2)} \cdots g_{t_m}^{\sigma(m)} g_{t_{j+1}}^{\sigma(m+1)}}$$

$$= \boxed{g_{t_{j+1}}^{+} g_{t_{j+2}}^{-} \cdots g_{t_m}^{\sigma(m-j-1)} g_{t_{j+1}}^{\sigma(m-j)}} g_{t_1}^{\sigma(m-j+1)} g_{t_2}^{\sigma(m-j+2)} \cdots g_{t_j}^{\sigma(m)} g_{t_1}^{\sigma(m+1)}$$

$$= g_{t_1}^{+} g_{t_2}^{-} \cdots g_{t_i}^{\sigma(i-1)} \boxed{g_{t_{j+1}}^{\sigma(i)} g_{t_{j+2}}^{\sigma(i+1)} \cdots g_{t_m}^{\sigma(i+m-j-1)} g_{t_{j+1}}^{\sigma(i+m-j)}} g_{t_{i+1}}^{\sigma(i+m-j+1)} \cdots g_{t_j}^{\sigma(m)} g_{t_1}^{\sigma(m+1)}.$$

Lemma 11. For AG_n, let $t_1, t_2, \ldots, t_m$ be distinct integers in $<n> - \{1, 2\}$. Then,

$$g_{t_1}^{+} g_{t_2}^{-} \cdots g_{t_j}^{\sigma(j-1)} \boxed{g_{t_{j+1}}^{\sigma(j+1)} g_{t_{j+2}}^{\sigma(j+2)} \cdots g_{t_m}^{\sigma(m)} g_{t_{j+1}}^{\sigma(m+1)}}$$

$$= g_{t_1}^{+} g_{t_2}^{-} \cdots g_{t_i}^{\sigma(i-1)} \boxed{g_{t_{j+1}}^{\sigma(i)} g_{t_{j+2}}^{\sigma(i+1)} \cdots g_{t_m}^{\sigma(i+m-j-1)} g_{t_{j+1}}^{\sigma(i+m-j)}} g_{t_{i+1}}^{\sigma(i+m-j+1)} \cdots g_{t_j}^{\sigma(m)}$$

$$= \boxed{g_{t_{j+1}}^{\sigma(l)} g_{t_{j+2}}^{\sigma(l+1)} \cdots g_{t_m}^{\sigma(l+m-j-1)} g_{t_{j+1}}^{\sigma(l+m-j)}} g_{t_1}^{\sigma(m-j+1)} g_{t_2}^{\sigma(m-j+2)} \cdots g_{t_j}^{\sigma(m)}.$$

When commuting with a barrel product that contains odd number of generators, the ordinary product should turn to its invert-signed product. The following example will illustrate the nesting of subproducts.

$$\Pi = \boxed{g_3^{+} g_4^{-} g_3^{+}} g_5^{+} g_6^{-}$$

$$= g_5^{-} g_6^{+} \boxed{g_3^{+} g_4^{-} g_3^{+}}$$

$$= g_5^{-} g_6^{+} \boxed{g_3^{-} g_4^{+} g_3^{-}}$$

$$= g_5^{-} \boxed{g_3^{-} g_4^{+} g_3^{-}} g_6^{-}$$

$$= g_5^{-} \boxed{g_3^{+} g_4^{-} g_3^{+}} g_6^{-}.$$

Definition 12. The *coupled ordinary product* is a concatenation of two *ordinary products* such that all the generators are of distinct-index and their boundary is of the same sign, not alternating signs.

For example, $g_3^{+} g_4^{-} g_5^{-} g_6^{+}$ is a coupled ordinary product.

Lemma 13. If Π_1 and Π_2 are two ordinary products and $\Pi_1\Pi_2$ is a coupled ordinary product,

$$\Pi_1\Pi_2 = \Pi_2^{\sigma(|\Pi_1|)}\, \Pi_1^{\sigma(|\Pi_2|)},$$

where $|\Pi_1|$ and $|\Pi_2|$ denote the numbers of generators contained in Π_1 and Π_2, respectively.

Definition 14. The *generalized ordinary product* is either an ordinary product or a coupled ordinary product.

Corollary 15. If some product of generators in AG_n, consists of k products of distinct sets of generators

$$\Pi = \Pi_1 \cdot \Pi_2 \cdots \Pi_i \cdot \Pi_k$$

and if at most one of these products is a generalized ordinary one while all other products are of barrel type, then the ordering of products Π_i in the overall product Π is arbitrary except that the signs of the generalized ordinary one may be obligated to invert.

Theorem 16. Every product of arbitrary number of generators in AG_n can be reduced to a product of $k \geq 1$ subproducts of distinct sets of generators

$$\Pi_m = \Pi_1 \cdot \Pi_2 \cdots \Pi_k$$

in which at most one subproduct is a generalized ordinary one and all others are of barrel type. Product Π_m cannot be further reduced, therefore it has the form of “minimal” product of generators.

Without loss of generality, we let $\prod_1$ be the generalized ordinary subproduct if it exists. Product $\prod_m$ will be referred to as minimal product and it actually represents the routing function in AG_n. The diameter of AG_n corresponds to the minimal product with the maximal number of generators. Since a barrel product has the property of repeating the first generator, the maximal length of the minimal product corresponds to the maximal number of barrel products in it.

Lemma 17. The maximal number k of subproducts in a minimal product from Theorem 16 is $\lceil (n-2)/2 \rceil$.

3. Node-disjoint routing scheme between two nodes

Here, we show how to construct a maximal set of disjoint paths between any two nodes of AG_n. Let us call the first two positions of a permutation *header positions* and the others *trailer positions*. In our algorithm, we use the constrained Algorithm 2 modified from Algorithm 1 that keeps some symbol (always 1 or 2) at the header positions and fixes some symbol (always 2 or 1) at a trailer position.

Algorithm 2. route2(p, q, x, y, l) { /* Given two nodes p and q in AG_n (or $(1\ 2)\cdot(AG_n)$), symbol y is at position l, finding a shortest path from p to q such that x will stays at header positions once it has moved to a header position and y is fixed at position l. */

$p' = p$; // the current node

while ($p' \neq q$) {

  if ($p'_1 \notin \{q_1, q_2\}$ and $p'_1 \notin \{x, l\}$) {

    $i = p'_1$;

    output(g_i^+);

    $p' = p' \check{\epsilon} g_i^+$;

    continue; // next loop

  }

  if ($p'_2 \notin \{q_1, q_2\}$ and $p'_2 \notin \{x, l\}$) {

    $i = p'_2$;

    output(g_i^-);

    $p' = p' \check{\epsilon} g_i^-$;

    continue;

  }

  Select a non-invariant i (i.e., $p'_i \neq q_i$) such that $i \neq l$ and $3 \leq i \leq n$;

  if ($p'_1 \neq x$) {

    output(g_i^+);

    $p' = p' \check{\epsilon} g_i^+$;

  } else { // $p'_2 \neq x$

output(g_i^-);

$p' = p' \circ g_i^-$;

}

} // end of while

} // end of algorithm

Without loss of generality, we focus on the routing method from node $p = p_1p_2 \ldots p_n$ to identity node I. Let $PATH(h)$ denote a path from p to I whose first hop generator is h. We show how to construct $2(n-2)$ node-disjoint paths $\{PATH(h) \mid h \in \Omega\}$. Assume $\prod_m$ is the minimal product of generators routing from p to I. Depending on whether positions 1 and 2 contain 1 or 2, three cases are distinguished:

1. $\{p_1, p_2\} = \{1, 2\}$;

2. $\{p_1, p_2\} = \{1, x\}$ or $\{2, x\}$, $x \neq 1, 2$;

3. $1, 2 \notin \{p_1, p_2\}$.

Case 1: $\{p_1, p_2\} = \{1, 2\}$

In this case, $\prod_m$ contains merely barrel products. By Lemmas 6 and 7, for each $\prod_i$ in $\prod_m$, $1 \leq i \leq k$, we can derive $2(|\prod_i| - 1)$ different forms of $\prod_i$'s, each begins with a distinct signed generator. Then, we can apply Lemma 10 to insert other products within the $\prod_i$'s to derive $2(|\prod_i| - 1)$ different sequences of generators. Each of the sequences can optimally sort

p to I. In this way, we have $2\sum_{i=1}^{k}(|\Pi_i|-1)$ different paths.

For each invariant symbol x, $x \neq 1, 2$, we can derive two sequences of generators $g_x^{+}\Pi_m$ $g_x^{\sigma(l+1)}$ and $g_x^{-}\Pi_m g_x^{\sigma(l)}$, where $l = |\Pi_m|$. They are both equivalent to Π_m by the above lemmas.

Totally, we have derived $2(n-2)$ different paths. It can be verified that these paths are all node-disjoint by the cancellation law of group algebra.

Case 2: $\{p_1, p_2\} = \{1, x\}$ or $\{2, x\}$, $x \div 1, 2$

Π_m will contain one ordinary product, say Π_1, without couple. In the cycle structure of p, there will be a cycle of one of the following forms:

$$(1\ x_1\ x_2\ x_3\ \ldots\ x_t);$$

$$(1\ x_1\ x_2\ x_3\ \ldots\ x_t\ 2);$$

$$(2\ x_1\ x_2\ x_3\ \ldots\ x_t);$$

$$(2\ x_1\ x_2\ x_3\ \ldots\ x_t\ 1).$$

However, the latter two can be transformed to the first two by conjugation (or relabeling) as follows:

$$\text{CONGUATE}(p \text{ to } I) \quad \Leftrightarrow \quad (1\ 2)\,p\,(1\ 2) \text{ to } (1\ 2)\,I\,(1\ 2).$$

We only demonstrate the routing scheme for the first form because the second form can be handled in a similar way in $(1\ 2)\cdot(AG_n)$.

Let C_1 denote the set of symbols $\{x_1, x_2, x_3, \ldots, x_t\}$ in the cycle representation. Define notation *prior*(x) as $p^{-1}(x)$ if $p^{-1}(x) \neq 1, 2$ and as $p^{-1}(p^{-1}(x))$ if $p^{-1}(x) \in \{1, 2\}$, $3 \leq x \leq n$. In the domain of C_1, *prior* is the permutation $(x_t\ x_{t-1}\ \ldots\ x_1)$. Depending on the sign of the first generator h, we have the following two subcases.

(2.a) $h = g_i^-$. That is, symbol 2 moves to position i in the first hop. The main idea is to fix 2 at position i within the path. In such a way, we can derive $(n-2)$ different paths. These paths are node-disjoint to each other because 2 is fixed at different positions. As an example, let p = 32451768. The cycle structure of p is (1 3 4 5) (6 7). Thus, $\prod_m$ may be $g_3^- g_4^+ g_5^- g_6^+ g_7^- g_6^+$. We have the following paths:

$g_3^- g_4^+ g_5^- g_6^- g_7^+ g_6^- g_3^+$, $g_4^- g_5^+ g_3^+ g_6^- g_7^+ g_6^- g_4^+$, $g_5^- g_3^- g_4^+ g_6^- g_7^+ g_6^- g_5^+$,

$g_6^- g_3^- g_4^+ g_5^- g_7^- g_6^+$, $g_7^- g_3^- g_4^+ g_5^- g_6^- g_7^+$, and $g_8^- g_3^- g_4^+ g_5^- g_6^- g_7^+ g_6^- g_8+$.

(2.b) $h = g_i^+$. That is, the first generator moves x_1 to position i. In this case, we always keep 2 at header positions. Therefore, the sign must alternate within such paths.

Let us at first select a generator g_w^+ that belongs to a barrel product if it exists. Let $\prod$ denote the barrel product containing g_w^+ or g_w^-. By Lemmas 6 and 7, we can transform $\prod$ to an equivalent barrel form $\prod' = [g_w^+ \ldots]$. By inserting other barrel products into $\prod'$ we can derive a product $\prod''$ such that the signs of generators are alternating. In case $i = x_1$, we generates the shortest path $PATH(g_{x_1}^+) = \prod_1 \prod''^{\sigma(|\prod_1|)}$ and $PATH(g_w^+) = \prod'' \prod_1^{\sigma(|\prod|'')}$ if there

exists any barrel product, or merely $PATH(g_{x_1}^{+}) = \prod_1$ otherwise. It can be observed that within $PATH(g_{x_1}^{+})$, x_1 is fixed at position x_1, 1 moves to a header position only when x_t been sorted, and once 1 has moved to a header position, 1 will be fixed at position w within the remaining subpath; Within $PATH(g_w^{+})$, 1 is fixed at position x_t, x_1 returns to a header position only when all the symbols not in $C_1 \cup \{1, 2\}$ have been sorted, and once x_1 has returned to a header position, x_1 will be fixed at position x_1 within the remaining subpath. In the above example, we have $PATH(g_3^{+}) = g_3^{+}g_4^{-}g_5^{+}g_6^{-}g_7^{+}g_6^{-}$ and $PATH(g_6^{+}) = g_6^{+}g_7^{-}g_6^{+}g_3^{-}g_4^{+}g_5^{-}$ if we select 6 as the w.

If $i = x_j$, $2 \le j \le t$, the routing scheme contains two phases. In the first phase, we move 1 to position $prior(i)$ while fixing x_1 at position i. Once 1 has entered position $prior(i)$, the second phase will start. In the second phase, we route in a shortest path to node $g_{prior(i)}^{+}$ while fixing 1 at position $prior(i)$, and finally apply $g_{prior(i)}^{-}$ to reach I. In the above example, $PATH(g_4^{+})$ is $g_4^{+}g_5^{-}g_3^{+}g_4^{-}g_6^{+}g_7^{-}g_6^{+}g_3^{-}$, in which 2 is always at a header position and 3 is fixed at position 4 before 1 enters position 3:

Phase 1: Fix x_1=3 at position i = 4

32451768g_4^{+} 254**3**1768g_5^{-}124**3**5768g_3^{+} 24**13**5768 g_4^{-} 32**1**45768 g_6^{+}…g_6^{+} 23**1**45678 g_3^{-} I.

Phrase 2: Fix 1 at position $piror(4) = 3$

$PATH(g_5^{+})$ is $g_5^{+}g_4^{-}g_5^{+}g_3^{-}g_6^{+}g_7^{-}g_6^{+}g_4^{-}$.

If i is not in C_1, the routing scheme also contains two phases. In the first phase, we move

1 to a header position while holding x_1 at position i. Once 1 has moved to the header position, we will enter the second phase by applying g_i^{σ}, σ is + or -, to move 1 to position i. In the second phase, we apply the constrained shortest path algorithm that fixes 1 at position i to node g_i^+ and apply finally g_i^- to reach I. In the above example, $PATH(g_6^+)$ is $g_6^+ g_5^- g_6^+ g_3^- g_4^+ g_5^- g_7^+ g_6^-$, in which symbol 2 is always at a header position and 3 is fixed at position 6 before 1 enters position 6:

Phase 1: Fix x_1=3 at position i = 6 Phase 2: Fix 1 at position i = 6

32451768g_6^+ 27451**3**68g_5^-12457**3**68g_6^+ 23457**1**68 g_3^- 42357**1**68 $g_4^+ \ldots g_7^+$ 26345**1**78 g_6^- I.

$PATH(g_7^+)$ is $g_7^+ g_5^- g_7^+ g_3^- g_4^+ g_5^- g_6^+ g_7^-$ and $PATH(g_8^+)$ is $g_8^+ g_5^- g_8^+ g_3^- g_4^+ g_5^- g_6^+ g_7^- g_6^+ g_8^-$.

It can be verified that the length of each path is no more than that of the shortest one by four. We now prove that the paths are node-disjoint.

1. The paths derived in (2.b) are node-disjoint to those in (2.a) because symbol 2 is always at a header position.
2. We prove that they are node-disjoint to each other by contradiction. Assume $PATH(g_x^+)$ and $PATH(g_y^+)$ coincide on some intermediate node p' for $x \neq y$. There are three different conditions:

   (1) $x, y \in C_1$.

   Without loss of generality, let $x \neq x_1$. Assume x_1 is at position x of p'. According to the routing scheme, p' is a first-phase node in $PATH(g_x^+)$ and 1 is either at position x_t or at a

header position. Because $x \neq y$, x_1 is not at position y. Since $PATH(g_{x_1}^{+})$ fixes x_1 at position x_1, we have $y \neq x_1$. Therefore, p' is a second-phase node in $PATH(g_y^{+})$ and 1 is at position $prior(y)$. We derive $prior(y) = x_t$. According to the definition of $prior$, $y = x_1$. It contradicts $y \neq x_1$.

Therefore, x_1 is not at position x of p'. It implies that p' is a second-phase node in $PATH(g_x^{+})$ and 1 is at position $prior(x)$. Since $x \neq y$, 1 is not at position $prior(y)$. In $PATH(g_y^{+})$, $y \neq x_1$, p' is a first-phase node and x_1 is at position y. That will lead to a contradiction as above. On the other hand, in $PATH(g_{x_1}^{+})$, if 1 is not at position x_t, 1 is either at a header position or at position $w \notin C_1$. It contradicts 1 being at position $prior(x) \in C_1$.

(2) $x, y \notin C_1$.

Without loss of generality, let $x \neq w$. Assume x_1 is at position x of p'. Thus, p' is a first-phase node in $PATH(g_x^{+})$, 1 is either at position x_t or at a header position. Because $x \neq y$, x_1 is not at position y of p'. Since x_1 is not at position y, p' is a second-phase node in $PATH(g_y^{+})$ for $y \neq w$, and 1 is at position y. It contradicts that 1 is either at position x_t or at a header position. We have $y = w$. In $PATH(g_w^{+})$, however, x_1 appears only at position w, position x_1, or header positions. It contradicts x_1 being at position x, $x \notin C_1$ and $x \neq w$.

Therefore, x_1 is not at position x of p'. It implies that p' is a second-phase node in $PATH(g_x^{+})$ and 1 is at position x, not y. Since 1 is not at position y, p' is a first-phase node

in $PATH(g_y^+)$ and x_1 is at position y if $y \neq w$. That will lead to a contradiction as above. On the other hand, within $PATH(g_w^+)$, 1 is fixed at position $x_t \in C_1$. It contradicts 1 being at position $x \notin C_1$.

(3) Either x or y is in C_1, but not both.

Without loss of generality, let $x \in C_1$. Assume x_1 is at position x of p'. Since x_1 is not at position y, p' is a second-phase node in $PATH(g_y^+)$ and 1 is at position y if $y \neq w$. On the other hand, if $y = w$, 1 is always at position x_t within $PATH(g_y^+)$. Since x_1 is at position x of p, p' is a first-phase node in $PATH(g_x^+)$ and 1 is either at some position $i \in C_1$, or at a header position if $x \neq x_1$. By checking the position of 1, we have $y = w$ and 1 is at position x_t if $x \neq x_1$. However, in $PATH(g_w^+)$, x_1 can appear only at position w, position x_1, or header positions. It contradicts x_1 being at position $x \in C_1$. Thus, we have $x = x_1$. Because 1 can appear only at position x_t, position w, or header positions in $PATH(g_{x_1}^+)$, we can also derive $y = w$ by checking the position of 1. However, $PATH(g_{x_1}^+)$ and $PATH(g_w^+)$ cannot coincide on an intermediate node according to group theory.

Therefore, x_1 is not at position x of p'. We have $x \neq x_1$ because $PATH(g_{x_1}^+)$ fixes x_1 at position x. So p' is a second-phase node in $PATH(g_x^+)$, and 1 is at position $prior(x)$. Since $x \neq x_1$, $prior(x) \neq x_t$. We have $y \neq w$ because $PATH(g_w^+)$ fixes 1 at position x_t. Since 1 is not at position y, p' is a first-phase node in $PATH(g_y^+)$ and x_1 is at position y, 1 is either at position x_t or at a header position. It contradicts 1 being at position $prior(x)$.

Therefore, all the $2(n-2)$ paths are node-disjoint.

We now discuss the special case $p = g_i^+$ or g_i^-, $3 \le i \le n$, i.e., there exists no barrel product and $t = 1$. Following our scheme, we have the maximal set of node-disjoint paths $\{g_i^+, g_i^- g_i^-, g_j^- g_i^- g_j^-, g_j^+ g_i^- g_j^+ g_i^+ g_j^+ \mid 3 \le j \le n \text{ and } j \ne i\}$ if $p = g_i^+$ and $\{g_i^-, g_i^+ g_i^+, g_j^+ g_i^+ g_j^+, g_j^- g_i^+ g_j^- g_i^+ g_j^- \mid 3 \le j \le n \text{ and } j \ne i\}$ if $p = g_i^-$.

Case 3: 1, 2 č $\{p_1, p_2\}$

The product $\prod_m$ will contain one coupled ordinary product, say $\prod_1$. The cycle structure corresponds to $\prod_1$ will be of one of the following forms:

$$(1\ x_1\ x_2\ x_3\ \ldots\ x_t)\ (2\ y_1\ y_2\ y_3\ \ldots\ y_s);$$

$$(1\ x_1\ x_2\ x_3\ \ldots\ x_t\ 2\ y_1\ y_2\ y_3\ \ldots\ y_s).$$

We only demonstrate the routing scheme for the former. Readers can handle the latter in a similar way in the dual graph $(1\ 2)\cdot(AG_n)$.

Let C_1 denote $\{x_1, x_2, x_3, \ldots, x_t\}$ and C_2 denote $\{y_1, y_2, y_3, \ldots, y_s\}$. Assume $\prod_1 = \prod_o \prod_c$, $\prod_o$ corresponds to C_1 and $\prod_c$ to C_2. As an example, let $p = 35412768$. The cycle structure of p is (1 3 4) (2 5) (6 7). The $\prod_m$ may be $g_3^+ g_4^- g_5^- g_6^+ g_7^- g_6^+$, $\prod_o = g_3^+ g_4^-$ and $\prod_c = g_5^-$. Depending on the sign of the first generator h, we distinguish the two subcases:

(3.a) $h = g_i^+$. That is, the first generator moves x_1 to position i. In this case, we always keep 2 either at its original position or at header positions. Precisely speaking, once 2 has

moved to a header position, we will keep 2 at header positions in the remaining path. Furthermore, we fix 1 at some trailer position while 2 is at a header position except the end.

In case $i = x_1$, we generates the shortest path: $PATH(g_{x_1}^{+}) = g_{x_1}^{+}$[route2($p \cdot g_{x_1}^{+}$, $g_{y_s}^{-}$, 1, 2, y_s)]$g_{y_s}^{+}$. In the path, 1 will be always at a header position once it has moved to a header position. In the above example, we may have the path: $g_3^{+}g_4^{-}g_6^{-}g_7^{+}g_6^{-}g_5^{+}$.

In case $i = x_j$, $2 \le j \le t$, the routing scheme contains two phases. In the first phase, we directly move 1 to a header position without moving x_1 (at position i) if 1 is not at a header position, and then move 1 to position $prior(i)$. Once symbol 1 has entered $prior(i)$, the second phase will start. In the second phase, we fix 1 at position $prior(i)$ except reaching I. In the above example, $PATH(g_4^{+})$ may be $g_4^{+}g_3^{-}g_4^{+}g_5^{+}g_6^{+}g_7^{-}g_6^{+}g_3^{-}$.

In case $i \notin C_1$, the routing scheme also contains two phases. In the first phase, we directly move 1 to a header position without moving x_1 if 1 is not at a header position, and then move 1 to position i. Once 1 has entered position i, the second phase will start. In the second phase, we fix 1 at position i except reaching I. In the above example, we may have the following paths:

$g_5^{+}g_4^{+}g_5^{-}g_3^{+}g_4^{-}g_6^{+}g_7^{-}g_6^{+}g_5^{-}$, $g_6^{+}g_4^{+}g_6^{-}g_3^{+}g_4^{-}g_5^{+}g_7^{+}g_6^{-}$,

$g_7^{+}g_4^{+}g_7^{-}g_3^{+}g_4^{-}g_5^{+}g_6^{+}g_7^{-}$, and $g_8^{+}g_4^{+}g_8^{-}g_3^{+}g_4^{-}g_5^{+}g_6^{+}g_7^{-}g_6^{+}g_8^{-}$.

(3.b) $h = g_i^{-}$. That is, the first generator moves y_1 to position i. In this case, we always

keep 1 either at its original position or at header positions. Precisely speaking, once 1 has moved to a header position, we will keep 1 at header positions in the remaining path. Furthermore, we fix 2 at some trailer position while 1 is at a header position except the end.

In case $i = y_1$, we generates the shortest path: $PATH(g_{y_1}^-) = g_{y_1}^-$[route2($p \cdot g_{y_1}^-$, $g_{x_t}^+$, 2, 1, x_t)]$g_{x_t}^-$. In the path, 2 will be always at a header position once it has moved to a header position. In the above example, we may have the path: $g_5^- g_6^- g_7^+ g_6^- g_3^+ g_4^-$.

In case $i = y_j$, $2 \le j \le s$, the routing scheme contains two phases. In the first phase, we directly move 2 to a header position without moving y_1 (at position i) if 2 is not at a header position, and then move 2 to position $prior(i)$. Once 2 has entered position $prior(i)$, the second phase will start. In the second phase, we fix 2 at position $prior(i)$ except reaching I.

In case $i \notin c_2$, the routing scheme also contains two phases. In the first phase, we directly move 2 to a header position without moving y_1 if 2 is not at a header position, and then move 2 to position i. Once 2 has entered position i, the second phase will start. In the second phase, we fix 2 at position i except reaching I. In the above example, we may have the following paths:

$g_3^- g_5^- g_3^+ g_4^+ g_5^+ g_6^- g_7^+ g_6^- g_3^+$, $g_4^- g_5^- g_4^+ g_5^- g_3^+ g_6^- g_7^+ g_6^- g_4^+$,

$g_6^- g_5^- g_6^+ g_5^- g_3^+ g_4^- g_7^- g_6^+$, $g_7^- g_5^- g_7^+ g_5^- g_3^+ g_4^- g_6^- g_7^+$, and

$g_8^- g_5^- g_8^+ g_5^- g_3^+ g_4^- g_6^- g_7^+ g_6^- g_8^+$.

Totally, we can construct $2(n-2)$ paths. Readers can verify that the length of each path is

no more than that of the shortest one by four.

Now, we prove these paths are node-disjoint except that $PATH(g_{x_1}^-)$ and $PATH(g_{y_1}^+)$ may coincide on one node. Suppose there are two paths $PATH(g)$ and $PATH(h)$ coincident on an intermediate node p'. We have following different conditions:

1. $g = g_i^+$ and $h = g_j^+$, $i \neq j$.

Assume x_1 is at position i of p'. According to the routing scheme, p' is a first-phase node in $PATH(g_i^+)$, and 1 is either at position x_t or at a header position. Thus, p' is also a first-phase node in $PATH(g_j^+)$. We derive $p'_j = x_1 = p'_i$. A contradiction occurs.

Therefore, x_1 is not at position i of p', p' is a second-phase node in $PATH(g_i^+)$, and 1 is at position i if $i \notin C_1$ or at position $prior(i)$ if $i \in C_1$. However, in $PATH(g_j^+)$, $j \neq x_1$, once 1 has moved from position x_t to a header position, 1 will move to position j or $prior(j)$ in the next hop and stay there. On the other hand, if $j = x_1$, once 1 has moved to a header position, 1 will stay at header positions in the remaining path of $PATH(g_j^+)$. Whether $j = x_1$ or not, 1 cannot appear at position $i \notin C_1$. We derive that 1 is at position $j = prior(i)$, $i \in C_1$ and $j \in C_1$. Since 1 is not at $prior(j)$ and $j \in C_1$, p' is a first-phase node in $PATH(g_j^+)$. Therefore, x_1 is at position j of p'. It contradicts 1 being at position j.

2. $g = g_i^-$, $h = g_j^-$ and $i \neq j$.

The proof is similar as above.

3. $g = g_i^+$, $h = g_j^-$.

If $i = x_1$, symbol 2 is always fixed at position y_s within $PATH(g_i^+)$ and p' is a first-phase node in $PATH(g_j^-)$. Furthermore, $PATH(g_{x_1}^+)$ fixes x_1 at position x_1 except the ends. However, x_1 never moves to position x_1 in the first phase of $PATH(g_j^-)$. Thus, we have $i \div x_1$. Similarly, we have $j \div y_1$.

Now, let us assume x_1 is not at position i of p' and y_1 is not at position j of p'. It implies p' is a second-phase node both in $PATH(g_i^+)$ and $PATH(g_j^-)$, 1 is at position $prior(i)$ if $i \in C_1$ or position i if $i \notin C_1$, and 2 is at position $prior(j)$ if $j \in C_2$ or j if $j \notin C_2$. But, in $PATH(g_i^+)$, once 2 has moved to a header position, 2 will stay at header positions and cannot be at $prior(j)$ if j in C_2 and j if $j \notin C_2$. A contradiction occurs.

Therefore, either x_1 is at position i of p' or y_1 is at position j of p'. Without loss of generality, let us assume x_1 is at position i of p'. It implies p' is a first-phase node in $PATH(g_i^+)$ and 1 is not at position $prior(i)$ if $i \in C_1$ and position i if $i \notin C_1$. Because in $PATH(g_i^+)$ we move 2 to a header position only if 1 has been fixed at the trailer position (position $prior(i)$ or i), 2 is at position y_s of p' unless $i = y_s$. In case $i = y_s$, 2 will stay at header positions and 1 will be fixed at position i within the remaining subpath of $PATH(g_i^+)$. Therefore, 2 cannot be at a position l, $l \div y_s$. That is, p' is also a first-phase node in $PATH(g_j^-)$ and y_1 is at position j.

Because in $PATH(g_i^+)$ we directly move 1 to a header position and then move 1 to position i or $prior(i)$, y_1 appears at position j unless $j = x_t$, the original position of symbol 1. We derive $j = x_t$. Similarly, we derive $i = y_s$.

$PATH(g_{y_s}^{+})$ and $PATH(g_{x_t}^{-})$ cannot coincide on another node because in $PATH(g_{y_s}^{+})$ once 1 has moved to a header position, 1 will immediately move to position y_s in the next hop and 2 will stay at header positions. On the other hand, in $PATH(g_{x_t}^{-})$ once 2 has moved to a header position, 2 immediately moves to position x_t in the next hop and 1 will stay at header positions. $PATH(g_{y_s}^{+})$ and $PATH(g_{x_t}^{-})$ only coincide on the second hop node, in which symbols 1 and 2 are both at header positions and y_1 at position x_t, x_1 at position y_s.

Nevertheless, we can easily adjust $PATH(g_{y_s}^{+})$ and $PATH(g_{x_t}^{-})$ to skip the coincident node by exchanging their second-phase subpaths because the product of the first generators in $PATH(g_{y_s}^{+})$ is $g_{y_s}^{+}g_{x_t}^{+}g_{y_s}^{-} = g_{x_t}^{-}g_{y_s}^{+}$ and the product of the first generators in $PATH(g_{x_t}^{-})$ is $g_{x_t}^{-}g_{y_s}^{-}g_{x_t}^{+} = g_{y_s}^{+}g_{x_t}^{-}$. Surely, they are node-disjoint to each other and node-disjoint to all the others. In the above example, we have $PATH'(g_{y_s}^{+}) = g_5^{+}g_4^{-}g_5^{-}g_3^{+}g_6^{-}g_7^{+}g_6^{-}g_4^{+}$ and $PATH'(g_4^{-})$ $= g_4^{-}g_5^{+}g_3^{+}g_4^{-}g_6^{+}g_7^{-}g_6^{+}g_5^{-}$.

For the overall routing algorithm, please refer to the appendix. A smart implementation of the routing algorithm can take $O(n^2)$ time to generate $\{PATH(h) \mid h \in \Omega\}$.

4. Concluding remarks

In this paper, we have demonstrated some algebraic properties of the generators in AG_n and the relationship between the barrel or ordinary products and the cycle structure. By confining the freedom of symbols 1 and 2, a simple scheme is provided for constructing a maximal set of node-disjoint paths between any two nodes in AG_n. Moreover, among the

disjoint paths, there is at least one shortest path and the length of longest one(s) is no more than that of a shortest path by four. Based on our result, further research can be focused on the following problems:

1.What is the maximal number of shortest node-disjoint paths between two nodes? How to construct them?

2.How to construct a set of node-disjoint paths from one node to arbitrary $2(n-2)$ nodes?

3.How to construct $2(n-2)$ edge-disjoint spanning trees?

References

[1] S. B. Akers, D. Harel, and B. Krishnamurthy. “The star graph: An attractive alternative to the n-cube.” In *Proceedings of the International Conference on Parallel Processing*, pp.393–400, 1987.

[2] S. B. Akers and B. Krishnamurthy. “Group graphs as interconnection networks.” In *Proceedings of the 14th International Conference on Fault Tolerant Computing*, pp. 422–427, 1984.

[3] Béla Bollobás, Extremal graph theory. 1978. Academic Press Inc.

[4] E. Cheng and M. J. Lipman, “Orienting Split-Stars and Alternating Group Graphs,” Networks, Vol. 35(2), pp. 139-144, 2000.

[5] W.-K. Chiang and R.-J. Chen, “On the Arrangement Graph,” Information Processing Letters, Vol. 66, pp. 215-219, 1998.

[6] S. P. Dandamudi. “On hypercube-based hierarchical interconnection network design. *Journal of Parallel and Distributed Computing*, Vol. 12, No. 3, pp.283–289, 1991.

[7] Q.-P. Gu and S. Peng, “Node-to-Set Disjoint Paths Problem in Star Graphs,” *Information Processing Letters*, Vol. 62, pp.201-207, 1997.

[8] Y. O. Hamidoune. “The connectivity of hierarchical Cayley digraphs.” *Discrete Applied Mathematics*, Vol. 37/38, pp.275–280, 1992.

[9] D. F. Hsu, “On Container Width and Length in Graphs, Group and Networks,” *IEICE Trans. Fundamentals Electronics, Information and Computer Science,* E77-A, Vol.4, pp.668-680, 1994.

[10] H.-S Hung and H-L Huang, “Broadcasting on the Alternating Group Graph”, In Proceedings of the 2001 National Computer Symposium, PCCU, Taipei, Taiwan, Dec. 2001, A085.

[11] Z. Jovanoviæ and J. Mišiæ “Fault tolerance of the star graph interconnection network.” *Information Processing Letters*, Vol.49, pp.145–150, 1994.

[12] J.-S. Jwo, S. Lakshmivarahan, and S. K. Dhall. “Characterization of node disjoint (parallel) paths in star graphs.” In *Proceedings of the 5th International Parallel Processing Symposium*, pp.404–409, April 1991.

[13] J.-S. Jwo, S. Lakshmivarahan, and S. K. Dhall. “A new class of interconnection networks based on the alternating group.” *Networks*, Vol.23, No.4, pp.315–326, July

1993.

[14] M. S. Krishnamoorthy and B. Krishnamurthy. “Fault diameter of interconnection networks.” *Comput. Math. Applic.*, Vol.13, No. 5/6, pp.577–582, 1987.

[15] C.-M. Lai and J.-J. Tsay, “Communication Algorithms on Alternating Group Graph,” In Proceedings of the Second Aizu International Symposium, Parallel Algorithms/Architecture Synthesis, pp.104-110, 1997.

[16] S. Latifi. “On the fault-diameter of the star graph.” *Information Processing Letters*, Vol. 46, pp.145–150, 1993.

[17] C.-T. Lin. “Fault Diameter of the Cayley Graphs Based on the Alternating Group,” In *Proceedings of the 2000 International Computer Symposium, Workshop on Computer Architecture,* National Chung Cheng University, Chiayi, Taiwan, pp.40-45, Dec. 6-8, 2000.

[18] Y. Rouskov and P. K. Srimani. “Fault diameter of star graphs.” *Information Processing Letters*, Vol. 48, pp.243–251, 1993.

[19] J-S Yang, T-Y Ho, Y-L Wang, and M-T Ko, “Applying Latin Square on Parallel Routing of Alternating Group Graphs,” In *Proceedings of the 2000 International Computer Symposium, Workshop on Algorithm,* National Chung Cheng University, Chiayi, Taiwan, Dec. 6-8, 2000.

Appendix

Algorithm 3. route3(a, b) { // Given two nodes a and b in AG_n,
// finding a maximal set of node-disjoint paths from a to b.
Construct the cycle structure of $p = b^{-1}a$;
if (p_1, $p_2 \in \{1, 2\}$) { // case 1: no ordinary product
if ((1 2) appears in the cycle structure) {
for ($3 \le i \le n$) {
output(g_i^+); route2($p \cdot g_i^+$, g_i^- , 1, 2, i); output(g_i^+); print(“,”);
output(g_i^-); route2($p \cdot g_i^-$, g_i^+ , 2, 1, i); output(g_i^-); print(“,”);
}
} else {
for ($3 \le i \le n$) {
output(g_i^+); route2($p \cdot g_i^+$, g_i^+ , 2, 1, i); output(g_i^-); print(“,”);
output(g_i^-); route2($p \cdot g_i^-$, g_i^- , 1, 2, i); output(g_i^+); print(“,”);
}
}
} // end of case 1
if ($\{p_1, p_2\} = \{1, x\}$ or $\{2, x\}$, $x \ne 1, 2$) { // case 2:
// an ordinary product without couple.
Compute the ordinary product $\prod_1$;
if (exists barrel product) {
Select j such that g_j^+ or g_j^- is in a barrel product;
} else {
$j = 0$;
}
$k = p^{-1}(1)$; // position of 1, i.e., x_t
$l = p(1)$; // x_1
if ($p_2 = 2$) {
for ($3 \le i \le$ n) { // fix 2 at i
output(g_i^-); route2($p \cdot g_i^-$, g_i^-, 0, 2, i);output(g_i^+); print(“,”);
} // end of for
// the paths that hold 2 at header positions
if (exists barrel product) {
// Generate the shortest paths
output($\prod_1 g_j^{\sigma(|\Pi_1|)}$); route2($p \cdot \prod_1 \cdot g_j^{\sigma(|\Pi_1|)}$, g_j^+, 2, 1, j); output(g_j^-); print(“,”);
output(g_j^+); route2($p \cdot g_j^+$, $\prod_1{}^{-1} \cdot g_j^{\sigma(|\Pi_1|)}$, 2, l, j); output($g_j^{\sigma(|\Pi_1|+1)}$);
output($\prod_1{}^{\sigma(|\Pi_1|)}$); print(“,”);

}
else { // no barrel product
// Generate the shortest path
output($\prod_1$); print(“,”);
} // end of if
for ($3 \le i \le n$) {
if ($i \in \{j, l\}$) continue; // has been processed above.
if ($i \in c_1$ and $t \ge 2$) {
if ($i \ne k$) {
output($g_i^+ g_k^- g_{prior(i)}^+$); // phase 1
route($p \cdot g_i^+ \cdot g_k^- \cdot g_{prior(i)}^+$, $g_{prior(i)}^+$, 2, 1, $prior(i)$); output($g_{prior(i)}^-$); // phase 2
} else { // 1 is at header position after first hop, move 1 to $prior(i)$ directly.
output($g_i^+ g_{prior(i)}^-$); // phase 1
route($p \cdot g_i^+ \cdot g_{prior(i)}^-$, $g_{prior(i)}^+$, 2, 1, $prior(i)$); output($g_{prior(i)}^-$); // phase 2
}
print(“,”);
} else if ($i \notin c_1$) {
output($g_i^+ g_k^- g_i^+$); // phase 1
route($p \cdot g_i^+ \cdot g_k^- \cdot g_i^+$, g_i^+, 2, 1, i); output(g_i^-); // phase 2
print(“,”);
} // end of if
} // end of for
} // end of if ($p_2 = 2$)
if ($p_1 = 2$) { // the dual case of conguation
for ($3 \le i \le n$) { // fix 2 at i
output(g_i^+); route($p \cdot g_i^+$, g_i^-) in $AG_n[2{:}i]$;output(g_i^+); print(“,”);
} // end of for
// the paths that holds 2 at header positions
if (exists barrel product) {
// Generate the adjusted paths
output($\prod_1 g_j^{s(|\prod_1|+1)}$); route2($p \cdot \prod_1 \cdot g_j^{s(|\prod_1|+1)}$, g_j^+, 2, 1, j); output(g_j^-); print(“,”);
output(g_j^-); route2($p \cdot g_j^-$, $\prod_1^{-1} \cdot g_j^{\sigma(|\prod_1|)}$, 2, l, j); output($g_j^{\sigma(|\prod_1|+1)}$);
output($\prod_1^{\sigma(|\prod_1|)}$); print(“,”);
}
else { // no barrel product
// Generate the shortest paths
output($\prod_1$); print(“,”);
} // end of if

for ($3 \le i \le$ n) {
if ($i \in \{j, l\}$) continue; // has been processed above.
if ($i \in c_1$ and $t \ge 2$) {
if ($i \ne k$) {
output($g_i^- g_k^+ g_{prior(i)}^-$); // phase 1
route($p \cdot g_i^- \cdot g_k^+ \cdot g_{prior(i)}^-$, $g_{prior(i)}^+$, 2, 1, $prior(i)$); output($g_{prior(i)}^-$); // phase 2
} else { // 1 is at header position after first hop, move 1 to $prior(i)$ directly.
output($g_i^- g_{prior(i)}^+$); // phase 1
route($p \cdot g_i^- \cdot g_{prior(i)}^+$, $g_{prior(i)}^+$, 2, 1, $prior(i)$); output($g_{prior(i)}^-$); // phase 2
}
print(“,”);
} else if ($i \notin c_1$) {
output($g_i^- g_k^+ g_i^-$); // phase 1
route($p \cdot g_i^- \cdot g_k^+ \cdot g_i^-$, g_i^+, 2, 1, i); output(g_i^-); // phase 2
print(“,”);
} // end of if
} // end of for
} // end of if ($p_1 = 2$)
if ($p_1 = 1$) { // c_1 contains 2
Conguate the subcase $p_2 = 2$.
}
if ($p_2 = 1$) {
Conguate the subcase $p_1 = 2$.
}
} // end of case 2
if (1, $2 \notin \{p_1, p_2\}$) {// case 3: $\prod_m$ contains a coupled ordinary product
$k_1 = p^{-1}(1)$; // position of 1, i.e., x_t
$l_1 = p(1)$; // x_1
$k_2 = p^{-1}(2)$; // position of 2, i.e., y_s
$l_2 = p(2)$; // y_1
if (1 and 2 are in the same cycle) { // transform to dual
dual = True;
Separate the cycle into two, c_1, c_2;
$p = (1\ 2) \cdot p$;
} else {
dual = False;
}
// Generate the two shortest paths

if (not dual) {
  output($g_{l_1}^+$); route2($p{\cdot}g_{l_1}^+$, $g_{k_2}^-$, 1, 2, k_2);output($g_{k_2}^+$); output(“,”);
  output($g_{l_2}^-$); route2($p{\cdot}g_{l_2}^-$, $g_{k_1}^+$, 2, 1, k_1);output($g_{k_1}^-$); output(“,”);
} else {
  output($g_{l_1}^+$); route2($p{\cdot}g_{l_1}^+$, $g_{k_2}^+$, 1, 2, k_2);output($g_{k_2}^-$); output(“,”);
  output($g_{l_2}^-$); route2($p{\cdot}g_{l_2}^-$, $g_{k_1}^-$, 2, 1, k_1);output($g_{k_1}^+$); output(“,”);
}
// Generate $PATH(g_i^+)$, $3 \le i \le n$, $i \ne x_1$
for ($3 \le i \le n$) {
  if ($i = l_1$) continue; // has been processed
  if ($i = k_2$) { // the adjusted path
    output($g_i^+ g_{k_1}^-$); // phase 1
    // phase 2
    if (not dual) {
      route2($p{\cdot}g_i^+{\cdot}g_{k_1}^-$, $g_{k_1}^-$, 1, 2, k_1); output($g_{k_1}^+$);
    } else {
      route2($p{\cdot}g_i^+{\cdot}g_{k_1}^-$, (2 1)$\cdot g_{k_1}^+$, 1, 2, k_1); output($g_{k_1}^-$);
    }
    output(“;”);
  }
  else if ($i \in c_1$) { // $t \ge 2$
    if ($i = k_1$) { // 1 is at a header position after the first hop,
      output($g_i^+ g_{prior(i)}^-$); // phase 1
      // phase 2
      if (not dual) {
        route2($p{\cdot}g_i^+{\cdot}g_{prior(i)}^-$, $g_{prior(i)}^+$, 2, 1, $prior(i)$); output($g_{prior(i)}^-$);
      } else {
        route2($p{\cdot}g_i^+{\cdot}g_{prior(i)}^-$, (2 1)$\cdot g_{prior(i)}^-$, 2, 1, $prior(i)$); output($g_{prior(i)}^+$);
      }
      output(“;”);
    } else {
      output($g_i^+ g_{k_1}^- g_{prior(i)}^+$); // phase 1
      // phase 2
      if (not dual) {
        route2($p{\cdot}g_i^+{\cdot}g_{k_1}^-{\cdot}g_{prior(i)}^+$, $g_{prior(i)}^+$, 2, 1, $prior(i)$); output($g_{prior(i)}^-$);
      } else {
        route2($p{\cdot}g_i^+{\cdot}g_{k_1}^-{\cdot}g_{prior(i)}^+$, (2 1)$\cdot g_{prior(i)}^-$, 2, 1, $prior(i)$); output($g_{prior(i)}^+$);
      }

output(“;”);
}
} else { // $i \notin c_1$
output($g_i^+ g_{k_1}^- g_i^+$); // phase 1
// phase 2
if (not dual) {
route2($p \cdot g_i^+ \cdot g_{k_1}^- \cdot g_i^+$, g_i^+ , 2, 1, i); output(g_i^-);
} else {
route2($p \cdot g_i^+ \cdot g_{k_1}^- \cdot g_i^+$, (2 1)$\cdot g_i^-$, 2, 1, i); output(g_i^+);
}
output(“;”);
}
} // end of for
// Generate $PATH(g_i^-)$, $3 \le i \le n$, $i \ne y_1$
for ($3 \le i \le n$) {
if ($i = l_2$) continue;
if ($i = k_1$) { // the adjusted path
output($g_i^- g_{k_2}^+$); // phase 1
// phase 2
if (not dual) {
route2($p \cdot g_i^- \cdot g_{k_2}^+$, $g_{k_2}^+$, 2, 1, k_2); output($g_{k_2}^-$);
} else {
route2($p \cdot g_i^- \cdot g_{k_1}^+$, (2 1)$\cdot g_{k_2}^-$, 2, 1, k_2); output($g_{k_2}^+$);
}
output(“;”);
}
else if ($i \in c_2$) { // $s \ge 2$
if ($i = k_2$) { // 2 is at a header position after the first hop,
output($g_i^- g_{prior(i)}^+$); // phase 1
// phase 2
if (not dual) {
route2($p \cdot g_i^- \cdot g_{prior(i)}^+$, $g_{prior(i)}^-$, 1, 2, $prior(i)$); output($g_{prior(i)}^+$);
} else {
route2($p \cdot g_i^- \cdot g_{prior(i)}^+$, (2 1)$\cdot g_{prior(i)}^+$, 1, 2, $prior(i)$); output($g_{prior(i)}^-$);
}
output(“;”);
} else {
output($g_i^- g_{k_2}^+ g_{prior(i)}^-$); // phase 1

```
            // phase 2
            if (not dual) {
               route2(p·g_i^-·g_k2^+·g_prior(i)^-, g_prior(i)^- , 1, 2, prior(i)); output(g_prior(i)^+);
            } else {
               route2(p·g_i^-·g_k2^+·g_prior(i)^-, (2 1)·g_prior(i)^+ , 1, 2, prior(i)); output(g_prior(i)^-);
            }
            output(";");
         }
      } else { // i ∉ c_1
         output(g_i^- g_k2^+ g_i^-); // phase 1
         // phase 2
         if (not dual) {
            route2(p·g_i^-·g_k2^+·g_i^- , g_i^- , 1, 2, i); output(g_i^+);
         } else {
            route2(p·g_i^-·g_k2^+·g_i^- , (2 1)·g_i^+ , 1, 2, i); output(g_i^-);
         }
         output(";");
      }
   } // end of case 3
} // end of algorithm
```